Reflex Computation:
Semantic Execution Blocks and Interface-Knowledge Placement

Working Draft

August 2026

Abstract

Language models can describe arithmetic procedures while still making exact execution errors. Conventional tool-use systems address this weakness by requiring the model to decide to call a calculator and serialize an invocation. We study bounded local calculator interfaces that act during generation. The original strict watcher recognizes an arithmetic suffix before an equals sign; the next iteration adds deterministic LaTeX/Unicode surface normalization and a fenced calculator block that executes only when its closing delimiter arrives. All assisted conditions share the same typed postfix micro-IR, exact-rational engine, resource bounds, and textual reinjection. After baseline-only adaptive selection, an XPU comparison used 16 arithmetic examples, 12 controls, and seven conditions. Exact-match accuracy was 43.8% for LLM-only, 62.5% for explicit tools, 75.0% for normalized reflex, and 12.5% for the original zero-shot block; normalized reflex gained five paired examples and lost none ($p = 0.0625$, unadjusted exact McNemar). We then developed block prompts on a separate split and froze a negative-first, positive-last two-shot protocol. On the same diagnostic examples, frozen blocks reached 100% valid execution and arithmetic accuracy with Qwen3-0.6B, but falsely blocked 2 of 12 controls. Qwen3-1.7B reached 93.8% execution and accuracy with zero false blocks, gaining nine and losing one pair against LLM-only ($p = 0.0215$). A Qwen3-4B six-item XPU smoke reached 100% execution with no false block, but is not a held-out estimate. Protocol-only rank-8 LoRA adapters were then trained for Qwen3-0.6B, SmolLM2-1.7B, and the approved official Gemma3-1B checkpoint. With a minimal instruction, all three adapters reached 100% block execution with zero false blocks; answers were 100%, 100%, and 93.8%, respectively. Gemma reached 100% answers when ICL was added, whereas its base ICL and normalized-runtime conditions executed only 31.2% and 25.0%. These developmental results favor a hybrid: store stable semantic selection and serialization in small adapter weights, keep exact execution and enforced result use in the runtime, and reserve context for cold-start or model-specific residual behavior. The sample remains too small for a general scaling or family claim. A frozen 120-item GSM8K transfer stress test then reversed the synthetic result: LLM-only and an unexecuted block prompt both reached 10.8% accuracy, generic compute 2.5%, automatic runtime 3.3%, and LoRA block 0.8%, while an oracle bounded-calculator ledger reached 75.8%. Automatic Rescue Rate was 0/6. Thus interface syntax learned on synthetic data did not supply the natural-language decomposition needed for public transfer. This motivates an additional NL-to-ASL approach. We implement a scope-capable Pratt parser, canonical CCIR lowering, deterministic arithmetic workspace, public dataset miner, local Codex skill, and provider-neutral teacher interface. Four public sources yield 10,623 normalized records with zero scope collisions and zero teacher-facing chop failures. A revised semantic skill and answer-blind Codex pass produce 98/150 execution-verified semantic programs; two explicitly rationale-assisted repair rounds raise coverage to 150/150 (100 GSM8K and 50 TAT-QA). These are Q1 compiler-data candidates with deterministic semantic lint, not manually reviewed gold. A semantic-pattern-grouped Q1 pilot then gives 0/25 final executed answers without examples, 3/25 with five-shot ICL, 4/25 with ten-shot ICL, and 5/25, 7/25, and 7/25 for rank-8 LoRA trained on nested 25-, 50-, and 100-program subsets. LoRA-100

plus three-shot ICL reaches 10/25 (40%; 95% Wilson 23.4–59.3%). Training on 100 originals plus 900 controlled variants improves lowerability and large-number robustness but reaches only 6/25 untouched answers. A relation-diversified expansion then freezes 350 additional training programs: the resulting 450 training rows contain 397 normalized signatures and have zero signature overlap with the unchanged 25-row development and test sets. This is a learning signal and a data checkpoint, not a robust semantic compiler. Training the same rank-8 QKVO adapter on all 450 training originals then reaches 11/25 untouched answers (44%), compared with 7/25 for 100 originals; the gain supports data/semantic coverage as the first bottleneck but is concentrated in TAT-QA and does not establish a robust compiler. A clause-local adapter trained on 1,403 executed-state transitions reaches only 7/25 answers in strict predicted-state closed loop; supplying gold prior state reaches 6/25. The small, negative oracle gap and high delta validity localize the remaining failure to semantic grounding and lost global context rather than state-error propagation. Restoring the full question while retaining local-delta prediction recovers 11/25, equal to whole-program synthesis but not better.

1 Status and Scope

This document reports an implemented next-iteration protocol, the preliminary pilot that motivated it, adaptive calibration, and one held-out comparison. The repository contains a dependency-free calculator/runtime path, deterministic dataset generation, JSONL traces and summaries, a scripted plumbing check, and an optional token-by-token Hugging Face adapter. Scripted checks are marked `empirical: false`; their expected perfect calculator behavior must not be cited as model evidence. The XPU pilot reported below is empirical but too small for model-family, scaling, or replicated performance claims. The held-out run is empirical and protocol-frozen, but it also has one checkpoint and one greedy seed; its condition ordering is evidence for replication, not a final architecture-level estimate. The public GSM8K slice is likewise a single-checkpoint developmental transfer test, not a benchmark-wide estimate. The ASL compiler pilot is also single-checkpoint and has only 25 untouched test programs; its component and robustness results are diagnostic Q1 evidence.

Paper 1 asks one question:

> When a language model naturally exposes a completed arithmetic expression during generation, does automatic bounded evaluation improve reliability or cost relative to model-only generation and explicit calculator calls?

The scope remains calculator only and deterministic syntax interfaces. The experiment does not test semantic recognition of prose arithmetic, learned routing, persistent symbolic memory, transactional state, Progressive Retrieval Attention (PRA), or native key/value access. Those mechanisms would make a positive result harder to attribute and a negative result harder to diagnose.

2 Hypotheses

H1: exact reliability. On arithmetic examples for which the strict event is correctly detected and normalized, reflex assistance increases exact-match accuracy relative to the same model without assistance.

H2: interface efficiency. Reflex or fenced-block assistance reduces model-authored invocation overhead relative to an explicit calculator condition while remaining competitive in answer quality and wall time.

H3: difficulty scaling. As operator count and operand width increase, reflex accuracy degrades more slowly than model-only and matched-prompt conditions. The relevant evidence is the condition-by-difficulty interaction or fitted scaling slope, not one easy benchmark cell.

H4: model-size leverage. Smaller models receive at least as much marginal accuracy benefit from correct reflex assistance as larger matched-family models. This is tested as a model-size-by-condition interaction, not inferred from separate unrelated checkpoints.

These hypotheses are conditional on detection and use. Oracle detection estimates interface headroom; engine correctness alone does not demonstrate that the model will expose the syntax or use the inserted result.

3 Mechanism

3.1 Strict generation-time event

The reflex recognizes an integer arithmetic suffix completed by one equals sign, for example

`The total is 37 * 48 =`

and inserts the exact result before later generated text:

`The total is 37 * 48 = 1776.`

The event is ordinary written arithmetic, not a bespoke API token. Detection is incremental at character granularity even when a tokenizer supplies multi-character fragments. Thus, if an equals sign occurs in the middle of a fragment, the result is inserted before the remainder of that fragment is processed.

The lexical detector accepts digits, whitespace, parentheses, and `+`, `-`, `*`, `/`, `//`, `%`, and `**`. It requires an operator-like symbol, balanced parentheses, and a digit or closing parenthesis before the equals sign. It suppresses double-quoted candidates and rejects candidates adjacent to letters, underscores, or decimal points. Lexical detection remains separate from normalization: a candidate such as `+ 2` is logged and then rejected because it has no binary operation.

3.2 Typed micro-IR

The normalizer parses the candidate in expression mode and accepts only integer constants, binary operations, unary plus, and unary minus. Calls, names, attributes, indexing, containers, comparisons, booleans, floats, and all other syntax are rejected. The parsed tree is converted to a canonical postfix instruction sequence such as

`PUSH 7; PUSH 5; ADD; PUSH 3; MUL.`

The Python parser is used only to construct and inspect syntax. The source is never compiled or executed. A separate stack interpreter executes the micro-IR using exact rational numbers.

3.3 Resource policy

Every event is bounded before or during evaluation. The default policy is shown in Table 1. Bounds are configuration recorded in each request, not hidden assumptions.

Division by zero, malformed IR, timeout, and resource exhaustion are typed failures. Failed results are traced but not inserted. Exponent and intermediate bit limits are checked before a large value can propagate through subsequent instructions.

Table 1: Default calculator safety and complexity bounds.

Bound	Default
Expression characters	256
AST nodes / depth	64 / 16
Digits per integer literal	64
Absolute integer exponent	12
Numerator or denominator size	512 bits
Stack items	64
Engine time budget	25 ms
Recognizer suffix buffer	512 characters

Table 2: Minimum experimental conditions.

Condition	Model instruction and capability	Purpose
LLM only	Return the exact answer; no calculator	Unassisted checkpoint baseline
Matched prompt	Work carefully and check operations; no calculator	Stronger prompting without external capability
Explicit tool	Model writes `<tool:calculator>EXPR</tool>` and receives the same engine result	Conventional model-authored invocation baseline
Reflex	Model is asked to write the expression followed by one equals sign; runtime detects any valid strict suffix	Proposed automatic condition
Normalized reflex	Reflex syntax plus deterministic LaTeX, Unicode, whitespace, and bracket normalization	Surface-robust automatic condition
Calculator block	Model writes one fenced calculator block; execution waits for the closing fence	Explicit boundary without an API schema
Oracle	Same generation instruction as reflex, but the detector accepts only the gold canonical expression	Detection/selection headroom control

3.4 Observable interrupt pipeline

Each intervention follows

$$\text{DETECT} \rightarrow \text{NORMALIZE} \rightarrow \text{ROUTE} \rightarrow \text{EXECUTE} \rightarrow \text{STATE UPDATE} \rightarrow \text{REINJECT}.$$

Every transition receives a run ID, sequence number, timestamp, status, candidate/request IDs, duration where meaningful, and structured details. The append-only micro-state stores the typed request, result, creation time, detector, and engine provenance. This is enough to separate detector, parser, engine, and reinjection failures without importing the later transactional-state design.

4 Conditions

All conditions use the same arithmetic examples, checkpoint revision, decoding budget, and calculator implementation when a calculator is available.

The explicit format is deliberately unambiguous and is not counted as a reflex. Its parser and the reflex parser converge on the same arithmetic micro-IR and engine, isolating invocation

style from calculator capability. A stronger native function-calling baseline should be added when the selected model/runtime exposes a stable function-calling interface; the textual baseline is the portable minimum.

5 Benchmark

5.1 Factorial arithmetic generators

The legacy deterministic generator crosses operator count with operand digit width. The separate `hard_arithmetic_v1` generator crosses at least four operator-count bands, four digit-width bands, and three structures: left chains, nested trees, and multiplication trees. It includes addition, subtraction, multiplication, safe exact-rational division, mixed operations, and long intermediates. Each cell uses identity-derived random seeds, and gold answers are produced by a separate exact-rational reference evaluator rather than the calculator implementation under test. The full hard grid contains

$$4 \text{ operator counts} \times 4 \text{ digit widths} \times 3 \text{ structures} = 48 \text{ cells}$$

before replicate and seed factors. Every generated item is accepted only after the independent oracle and bounded calculator agree.

This synthetic benchmark isolates scaling and component correctness; it does not establish ecological prevalence. A later extension may add held-out natural model traces, but only after freezing annotation rules for whether a generated prefix contains a valid strict event.

5.2 Non-trigger and adversarial controls

Controls include prose containing numbers, ranges, parentheses, quoted arithmetic, decimal-looking versions, variable equations, and code-like equality. Additional engine tests cover division by zero, excessive exponents and values, disallowed calls/names/indexing, and unary-only candidates. Controls are scored for false intervention and component behavior, not arithmetic answer accuracy.

5.3 Model and seed sweep

The primary manifest pins one matched Qwen3 family at 0.6B, 1.7B, and 4B parameters and uses seeds 17, 29, and 41. Repository commit hashes are stored in the configuration and copied into run manifests. The model list is an initial controlled family, not a claim of model-family generality. A second family is required before making architecture-general claims.

Generation is greedy in the initial adapter so seed variability mainly reflects future sampled configurations and experimental plumbing; if sampling is enabled, temperature, top-p, and all random states must be pinned. The correctness-first adapter recomputes the full prefix per generated token after reinjection. This is intentionally inefficient but semantically transparent. Optimized KV-cache interception is outside Paper 1 and must not be silently credited to it.

6 Metrics and Analysis

End task. Report exact rational answer accuracy with 95% confidence intervals. If a model states a final answer after receiving a calculator result, score the final claim, not merely the correct engine value. Report correction, override, and result-use rates separately.

Components. Report intended-expression exposure, candidate recognition, full-expression selection, normalization acceptance/correctness, engine success/correctness, reinjection success, result use, and later override separately. Also report false-intervention rate, intervention count, and state-item count. Oracle differences localize missed or malformed interface events.

Cost. Report prompt, model-generated, and reinjected tokens separately; model calls; engine and end-to-end wall time; and serialized trace/state growth. Do not present the correctness-first Hugging Face adapter's full-prefix recomputation as a production latency estimate.

Scaling and inference. Report every operator-count by digit-width cell for each model, condition, and seed. Estimate condition-by-difficulty and condition-by-model-size interactions, with paired comparisons because all conditions share examples. Predeclare one primary endpoint and correct or label exploratory multiple comparisons. Preserve raw predictions and event traces so component errors can be reclassified.

7 Frozen Next-Iteration Protocol

The current interface freeze is versioned as `paper1_hard_interfaces_v2_concise`. It retains `strict_arithmetic_v1` unchanged and adds `normalized_arithmetic_v1`, `surface_normalizer_v1`, and `calculator_block_v1`. The normalizer maps allowlisted multiplication, division, Unicode-minus, bracket, and spacing aliases before delegating to the existing `ccpu.arithmetic.postfix.v1` parser. Unknown commands, prose, variables, quoted arithmetic, and quoted or inline fence examples remain inert.

The block grammar is a line-anchored Markdown-like fence named `calculator`. Its contents are collected incrementally, but no candidate is emitted until the closing fence is complete. Thus a valid intermediate line cannot execute prematurely. At close, the same surface normalizer and bounded calculator are used, and a compact typed result is inserted.

The adaptive pilot runs only the LLM-only condition over all 48 cells. Cell selection uses aggregate exact match to choose regimes near 95%, 65%, 30%, and 0%; it never examines assisted outcomes. Once those cells are selected, their definitions, prompts, grammar, normalizer, and diagnostics remain frozen. A distinct dataset seed then generates the held-out comparison across all seven conditions.

Every arithmetic prediction records expression exposure, candidate recognition, full-expression selection, normalization, execution, reinjection, result use, and later override independently. Cost records include prompt, generated, and reinjected tokens, model calls, wall and engine time, serialized trace/state bytes, and deterministic invocation-delimiter characters. Scripted smoke output is marked `empirical: false` and is plumbing evidence only.

7.1 Adaptive calibration status

Two baseline-only XPU calibrations exposed output censoring before any assisted condition was run. Both used Qwen3-0.6B, 96 arithmetic examples over 48 cells, two nominal generation seeds, and a 48-token ceiling, for 192 generations each. Under the original prompt, all 192 generations reached the ceiling and 184 had no scorable final answer. A uniformly applied concise-answer prompt reduced cap hits to 170 and missing answers to 156, but both versions yielded only 3.125% overall exact match per seed. The concise run had 46 cells at 0%, one at 50%, and one at 100%, so it cannot supply the four intended difficulty bands.

Table 3: Held-out exact-match accuracy. Intervals are 95% Wilson intervals over 16 arithmetic examples. Paired gains/losses and unadjusted exact McNemar p values compare each condition with LLM-only.

Condition	Accuracy [95% CI]	Gains–losses	Exact p
LLM only	0.438 [0.231, 0.668]	—	—
Matched prompt	0.500 [0.280, 0.720]	1–0	1.000
Explicit tool	0.625 [0.386, 0.815]	3–0	0.250
Strict reflex	0.375 [0.185, 0.614]	0–1	1.000
Normalized reflex	0.750 [0.505, 0.898]	5–0	0.0625
Calculator block	0.125 [0.035, 0.360]	2–7	0.180
Oracle	0.875 [0.640, 0.965]	7–0	0.0156

These runs are preserved as empirical calibration failures, not treated as the hard-arithmetic comparison. Since greedy argmax decoding produced identical text for every one of the 96 cross-seed pairs, the revised calibration removes the redundant generation seed, broadens the grid downward to one–four operand digits and one–four operators, uses two independently generated examples per cell, and raises the uniform ceiling to 96 tokens.

The revised calibration completed 96 baseline-only generations with 30.2% overall exact match; 78 generations had a scorable final answer. Across 48 cells, 9 reached 100% accuracy, 11 reached 50%, and 28 reached 0%. We froze four whole cells with complete answer coverage and no cap hits: one operator/one digit with a left chain (100%), two operators/two digits with nesting (50%), one operator/three digits with a multiplication tree (50%), and one operator/four digits with a left chain (0%). The two 50% cells are explicitly coarse approximations to the overlapping moderate and transition targets.

The selection record stores source dataset and prediction hashes and states that no assisted outcome was consulted. A separately seeded held-out dataset uses four examples per selected cell, cycling ASCII, LaTeX, Unicode, and bracket surfaces, plus all 12 adversarial controls. The held-out model run remains ungenerated at this protocol-freeze point; the results below were produced only after that dataset, selection record, and interface code were committed.

8 Held-Out XPU Comparison

8.1 Setup and end-task result

The held-out run used Qwen3-0.6B at revision `c1899de`, PyTorch 2.13.0+XPU, one greedy generation seed (17), and a uniform 96-token ceiling. Four newly generated examples in each of the four frozen cells gave 16 arithmetic examples; 12 controls were crossed with the same seven conditions. All 196 prediction rows report `empirical: true` and `device: xpu`.

Normalized reflex was numerically the strongest non-oracle condition, improving by 31.25 percentage points over LLM-only and 12.5 points over explicit tools. The five paired gains and zero losses are suggestive but do not reach 0.05 in the unadjusted exact test; no multiplicity correction is applied. Strict reflex lost one paired example and gained none. The four held-out cells contain only four examples each and do not show a stable monotone difficulty interaction: normalized-reflex cell accuracies were 1.00, 0.25, 1.00, and 0.75. H1 is therefore unsupported for the original strict watcher, while the normalized variant is a replication candidate rather than a confirmed claim.

Table 4: Held-out assisted-interface diagnostics over arithmetic examples. Selection is correctness conditional on an accepted request; use is conditional on successful execution. All conditions had zero false interventions on 12 controls, and engine correctness was 1.0 whenever executed.

Condition	Exposure	Recognition	Selection	Normalize/execute	Use	Override
Explicit tool	0.875	1.000	1.000	0.625	1.000	0.000
Strict reflex	0.625	0.313	0.600	0.313	0.800	0.200
Normalized reflex	0.750	0.750	1.000	0.750	1.000	0.000
Calculator block	0.375	1.000	1.000	0.125	1.000	0.000
Oracle	—	1.000	1.000	1.000	0.875	0.125

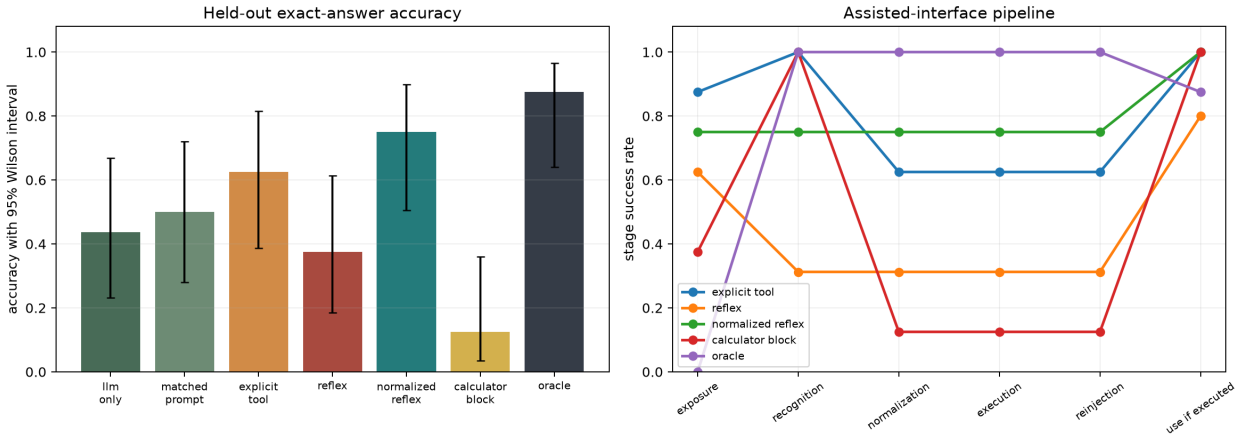

Figure 1: Held-out accuracy (left) and assisted-interface stage decomposition (right). Exposure is model-stream exposure and is not applicable to oracle priming. Use is conditional on execution, which explains high use despite low block normalization.

8.2 Interface decomposition

Normalized reflex reached and correctly selected the full expression in 12 of 16 cases; all 12 exact results were reinjected, used, and not overridden. Strict reflex executed five candidates, but only three represented the intended full expression. Explicit tool calls were recognized in every arithmetic generation, but six arguments failed the strict parser, primarily because the model copied LaTeX or Unicode notation.

The semantic block result falsifies H3 for the tested zero-shot prompt. All 16 closing fences were recognized, but 14 blocks contained the literal placeholder `EXPRESSION` or mixed instructions/prose with arithmetic. Only two blocks normalized and executed; both engine results were exact and used. Thus the block boundary prevented premature execution, but its demonstration-style prompt elicited invalid block contents. This is a generation/normalization failure, not a calculator or closing-fence detection failure.

Cost. Mean generated tokens were 43.1 (LLM-only), 46.1 (matched), 19.2 (explicit), 47.9 (strict), 22.4 (normalized), 32.7 (block), and 15.6 (oracle). Mean wall times followed the correctness-first decoder rather than production serving: 8.0, 8.8, 3.2, 9.6, 4.0, 5.9, and 2.7 seconds, respectively. Normalized reflex used fewer delimiter characters than explicit calls, but it did not beat explicit tools on every measured cost: explicit generated fewer tokens and was faster. H2 is therefore not supported as a quality–cost frontier claim.

Table 5: Frozen ICL-G calculator blocks. The 0.6B and 1.7B rows use 16 arithmetic and 12 control examples; the 4B row is a four-arithmetic/two-control smoke only.

Model	Phase	Execute	Semantic	Answer	False controls	Use
Qwen3-0.6B	held-out diagnostic	1.000	1.000	1.000	2/12	1.000
Qwen3-1.7B	held-out diagnostic	0.938	0.938	0.938	0/12	1.000
Qwen3-4B	smoke	1.000	1.000	1.000	0/2	1.000

8.3 Initial evidence-gate decision

At that point the decision was **no-go for Paper 2**. The original strict reflex does not improve held-out accuracy, the block interface fails zero-shot content elicitation, and the normalized-reflex advantage is unreplicated with $p = 0.0625$ over 16 pairs. A next Paper 1 replication should hold `surface_normalizer_v1` fixed, increase examples and generation seeds under sampling or add a second checkpoint family, and predeclare normalized reflex versus both LLM-only and explicit tool as primary comparisons. Block prompting or lightweight training is now eligible for a separate development split, but must not be tuned on this held-out set.

9 Few-Shot Blocks and Capability Diagnostic

9.1 Endpoint audit and prompt development

The original exact-match labels are preserved. A parallel, condition-independent endpoint extractor recognizes terminal forms such as `Response: N` without using condition identity. Under this audit the original zero-shot block's answer accuracy changes from 12.5% to 43.8%, while normalized reflex remains 75.0%. This is an extraction correction only: valid zero-shot block execution remains 12.5%, so the protocol conclusion is unchanged.

We tested six stronger block prompts on a separate developmental split containing four arithmetic examples and four controls. Concrete and verbatim-copy examples made arithmetic execution reliable but initially caused false blocks on all four controls; a positive/negative variant suppressed false blocks but also suppressed arithmetic blocks. A diagnostic with fully seeded fences isolated payload copying. The final order control, ICL-G, puts a negative demonstration first and a concrete positive block last. It achieved 100% semantic payload equivalence, execution, result use, and answer accuracy with no developmental false block. Exact-string payload match was 0% because the model removed redundant parentheses; the bounded parser established semantic equivalence. We froze this prompt as `paper1_calculator_block_icl_v2_order_control` before cross-model runs.

9.2 Matched-family XPU results

Table 5 compares the frozen prompt without per-model tuning. Qwen3-0.6B used the original held-out arithmetic/control set. The 1.7B checkpoint used all seven conditions over the same 28 examples, producing 196 rows. The 4B result is only the predeclared four-arithmetic/two-control smoke gate.

For 0.6B, ICL-G gained nine arithmetic examples and lost none against the preserved LLM-only run ($p = 0.0039$); however, two false blocks fail the safety part of the frozen gate. For 1.7B, it gained nine and lost one ($p = 0.0215$), while normalized reflex gained six and lost one ($p = 0.125$). The sole 1.7B block failure omitted the wrapper and emitted a bare expression. The endpoint

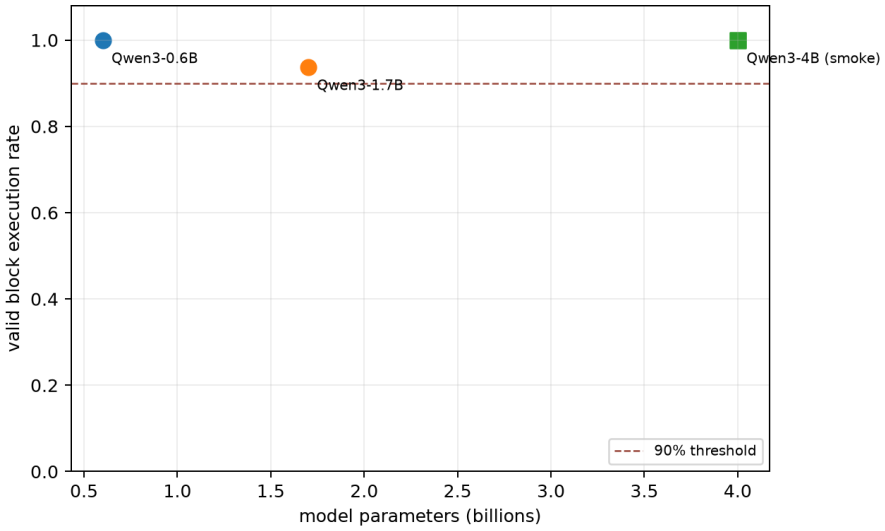

Figure 2: Valid execution under the one frozen ICL prompt. Circles are 16-example held-out diagnostics; the square is a four-example smoke and must not be read as a comparable estimate. Parameter count is descriptive, not a fitted scaling law.

audit leaves ICL-G at 93.8%. The unexpectedly low 1.7B oracle score (12.5% originally, 25.0% under endpoint rescoring) reflects model integration after seeded results, not calculator failure; it is retained rather than normalized away.

Family coverage and runtime. Official Llama 3.2 and Gemma checkpoints require manually accepted Hugging Face licenses. No authenticated token was available during this matched-family sweep, so four configured checkpoints were skipped and no community mirror was substituted; this capability-scaling phase is therefore Qwen-only. Approved Gemma access was supplied later for the adapter-placement phase below. Qwen3-1.7B peaked at 3.57 GB XPU memory and its full run took 3074 seconds. Qwen3-4B loaded and completed the smoke at 8.21 GB peak memory in 494 seconds. The block, accuracy/interface, token–latency, and failure plots are generated from one hashed comparison configuration and machine-readable table.

9.3 Current evidence and LoRA gate

The new result reopens the original interface gate: a frozen semantic block is reliable on the 1.7B diagnostic and clean in the 4B smoke. This is sufficient to motivate later heterogeneous-interface work, while confirmatory Paper 1 claims remain gated on a larger untouched set and additional families. It also places the LoRA question in Regime A: can 0.6B retain its perfect arithmetic protocol while eliminating false control blocks? This asks where interface knowledge should live: transiently in context, persistently in weights, or deterministically in the runtime.

Frozen adapter protocol. The minimal protocol-only experiment uses 80 arithmetic and 80 negative-control requests; a disjoint development set has 20 of each. Arithmetic targets contain only the requested expression inside a calculator fence, never its answer. They cover three–six digit operands, nesting, mixed operations, exact division, long intermediates, and ASCII, LaTeX, Unicode, and bracket surfaces. Controls cover versions, dates, ranges, quoted arithmetic, code,

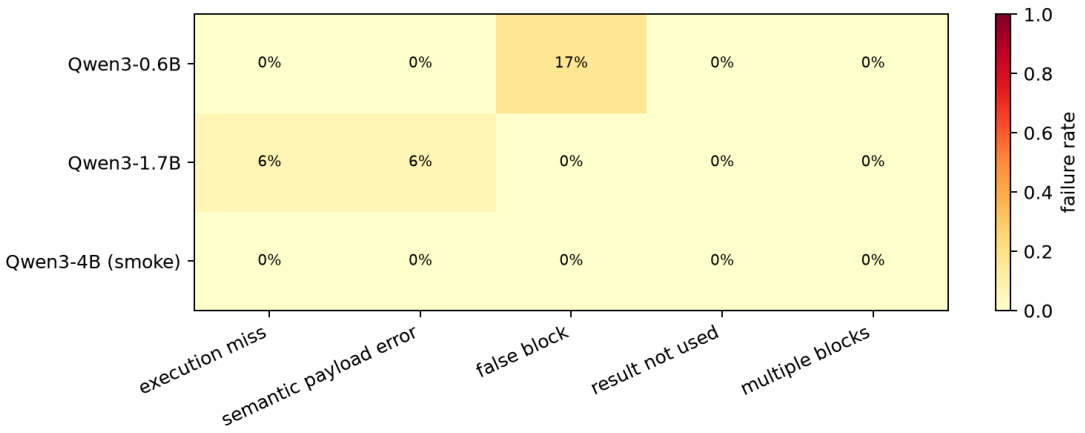

Figure 3: Block failure decomposition. The small model's remaining error is control discrimination, whereas the 1.7B model has one arithmetic protocol miss.

Table 6: Qwen3-0.6B interface-knowledge placement on 16 arithmetic examples and 12 controls. Execute is valid arithmetic execution; false is the number of control interventions. Tokens are means over all 28 examples.

Placement/condition	Execute	Answer	False	Prompt tok.	Output tok.
Base + minimal	0.000	0.000	0/12	86.6	17.2
Context: base + ICL-G	1.000	1.000	2/12	113.8	18.9
Weights: LoRA + minimal	1.000	1.000	0/12	86.6	14.9
Weights + context: LoRA+ICL	1.000	1.000	4/12	113.8	19.2
Runtime: normalized reflex	0.750	0.750	0/12	61.0	22.4

identifiers, and variable-bearing equations. A mechanical audit finds no exact expression or operand overlap among training, development, and the 16-example held-out benchmark.

The XPU training gate completed PEFT forward, backward, and AdamW steps on all three models. The frozen stack adds PEFT 0.20.0 and Accelerate 1.14.0 to PyTorch 2.13.0+XPU and Transformers 4.57.6. LoRA rank is 8 over attention projections, with two epochs and assistant-target-only loss. Qwen3-0.6B is primary. The predeclared ungated second family is the official Apache-2.0 HuggingFaceTB SmolLM2-1.7B-Instruct checkpoint, not a community mirror. A later authenticated run uses the approved official Gemma3-1B checkpoint at pinned revision `dcc83ea`; Llama 3.2 access remains unavailable.

Evaluation compares base+frozen ICL (context), adapter+minimal instruction (weights), and base+normalized reflex (runtime), with base+minimal and adapter+ICL controls. It reports protocol decomposition, false blocks, answer accuracy, prompt/output tokens, training parameters/tokens/time, memory, and inference overhead.

Qwen3-0.6B adapter result. The primary adapter trained in 394.5 seconds on XPU, with 2,293,760 trainable parameters (0.383% of the model), 7,362 assistant target tokens over two epochs, and 1.48 GB peak memory. Mean training loss fell from 0.0526 to 0.000032 and development loss from 0.000050 to 0.000021. The serialized adapter is 9.2 MB.

Base+minimal emitted no valid arithmetic block, so the instruction alone does not explain the adapter result. LoRA+minimal produced 16/16 semantically correct and executable payloads,

used every result, answered all 16 correctly, and opened no block on 12 controls. Payload strings differed harmlessly on 14 cases, primarily by removing redundant parentheses. The adapter adds about 9.2 MB of resident model memory; measured peak inference memory increased from 1.28 to 1.42 GB. Its minimal prompt uses 27.1 fewer tokens than ICL-G and generates four fewer tokens on average.

Context and weights are not additive here. Base+ICL-G had two false blocks; LoRA+ICL-G had four, including quoted arithmetic, quoted fences, pseudocode, and quoted code. Thus the primary Qwen result favors storing the learned semantic selection/serialization contract in adapter weights with a minimal explicit contract, while retaining deterministic execution in the runtime. The normalized runtime remains safer without training but has only 75% expression exposure and execution. No result implies that LoRA learned arithmetic: every answer still comes from the unchanged bounded calculator.

Cross-family replication and placement answer. SmolLM2-1.7B used the identical data and optimization schedule. Its adapter has 3,145,728 trainable parameters (0.183%), is 12.6 MB, used 7,542 target tokens, trained in 768.0 seconds, and peaked at 3.59 GB. Training/development loss changed from 0.1287/0.0049 after epoch one to 0.0013/0.00015 after epoch two.

Base+minimal again emitted 0/16 valid blocks. Base+ICL-G executed 15/16, answered 14/16, and falsely blocked one quoted-fence control. LoRA+minimal executed and answered 16/16, used every exact result, and had 0/12 false blocks. LoRA+ICL-G also answered 16/16 but retained the quoted-fence false block and emitted multiple blocks twice. Normalized runtime executed 15/16 with no false intervention but answered 13/16 because exact result integration was imperfect.

Gemma3-1B used the same data, schedule, and rank, with its native BF16 dtype. Its 1,490,944 trainable parameters (0.149%) serialize to 6.0 MB; training used 7,362 target tokens, took 365.0 seconds, and peaked at 2.52 GB. Training/development loss changed from 0.1654/0.000094 after epoch one to 0.000069/0.000021 after epoch two. Base+minimal emitted 0/16 valid blocks. Base+ICL-G executed 5/16 and falsely blocked one control; normalized runtime executed 4/16 and also had one false intervention. LoRA+minimal executed 16/16 with 0/12 false blocks but copied one expression incorrectly, yielding 15/16 answers. LoRA+ICL-G recovered 16/16 answers and retained 0/12 false blocks. Oracle detection executed all 16 exact calculations, but Gemma used none of the reinjected results, demonstrating that runtime enforcement cannot stop at detection, execution, and reinjection.

The manual autoregressive backend originally stopped only on the tokenizer's single EOS identifier. Gemma declares both EOS and `<end_of_turn>` in its model generation configuration. Corrected generation honors the full set {1,106}; the corrected base and adapter summaries are the only Gemma sources in this analysis. Earlier `v1` diagnostic runs are retained but excluded.

The answer is therefore not one location. Stable semantic selection and block serialization are most reliable in weights under a short explicit contract in all three models. New or changing interfaces can begin in context without training, but demonstrations recur on every request and produced more false interventions; residual context repaired Gemma's one adapter copying error but was redundant or harmful for Qwen and SmolLM. Parsing, resource bounds, exact arithmetic, reinjection, and enforced result use must remain in the runtime. This division stores policy in weights and mechanism in the runtime, with context serving development, cold start, and measured model-specific residuals. It is a developmental design result, not evidence that all interface knowledge should be fine-tuned.

Table 7: Primary placement comparison. Recurring tokens sum mean prompt and generated tokens. All rates use 16 arithmetic and 12 control examples.

Model	Placement	Interface	Answer	False	Tokens
Qwen3-0.6B	Context	1.000	1.000	0.167	132.7
	Weights	1.000	1.000	0.000	101.5
	Runtime	0.750	0.750	0.000	83.5
SmolLM2-1.7B	Context	0.938	0.875	0.083	184.2
	Weights	1.000	1.000	0.000	127.6
	Runtime	0.938	0.813	0.000	96.6
Gemma3-1B	Context	0.312	0.375	0.083	131.2
	Weights	1.000	0.938	0.000	102.1
	Runtime	0.250	0.312	0.083	86.8

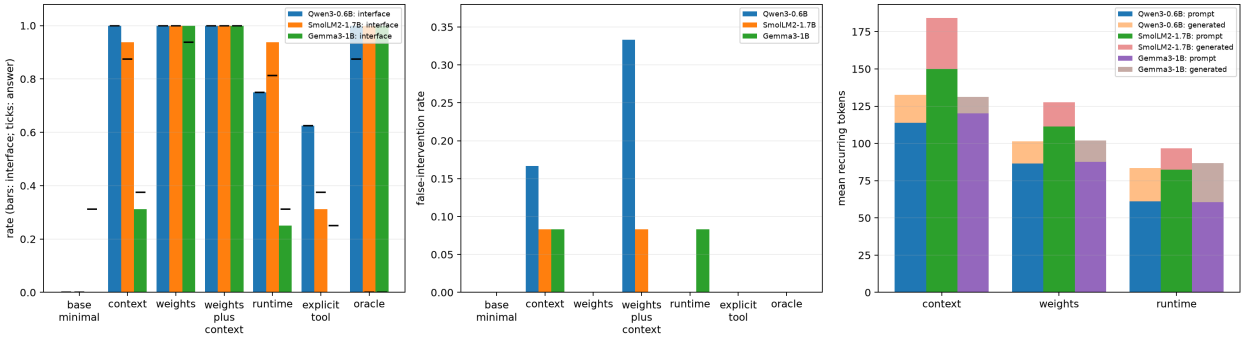

Figure 4: Interface success and answer accuracy (left), false interventions (center), and recurring prompt-plus-output tokens (right). Adapter+minimal has perfect execution and observed control safety in all three families; Gemma needs residual ICL to recover its one copied-expression error.

10 Public GSM8K Transfer Stress Test

10.1 Frozen protocol

GSM8K contains linguistically diverse grade-school word problems with annotated multi-step arithmetic rationales [6]. We selected 120 test examples by the lowest precomputed selection key within three registered strata: 40 with at most two annotated operations, 40 with three or four, and 40 with five or more. The selection artifact redistributes only identifiers, labels, difficulty metadata, and content hashes. Questions are materialized from the pinned source at run time and rejected if their hashes change.

Qwen3-0.6B at revision `c1899de` was evaluated once with greedy seed 23011, FP16 XPU, thinking disabled, and a 160-token ceiling. Seven conditions share the same selected IDs: LLM-only, matched ICL, voluntary generic `__compute`, base calculator block, normalized automatic runtime, oracle calculator ledger, and the retained rank-8 calculator-block LoRA. Generic compute and automatic runtime use the same bounded calculator. The oracle path does not trust source-side arithmetic values: it normalizes and re-executes every supported annotated expression through that calculator before constructing the ledger. Of 425 annotated operations, 382 (89.9%) are supported; 113/120 rows have at least one supported operation and 87/120 have full trace coverage.

The primary endpoint is exact final-answer accuracy. Scorable rate records whether the model emits a parseable endpoint. Assistance rate requires at least one accepted execution, and registered-

Table 8: Frozen Qwen3-0.6B GSM8K transfer test (120 examples). Op. recall is over calculator-supported annotated operations. Latency is the mean model wall time; interrupt conditions use correctness-first full-prefix decoding and are not production serving estimates.

Condition	Accuracy	Scorable	Assist	Op. recall	Tokens	ms
LLM only	0.108	1.000	0.000	0.000	18.7	2,776
Matched ICL	0.033	1.000	0.000	0.000	10.7	1,679
Generic compute	0.025	1.000	0.950	0.000	39.8	5,963
Base block prompt	0.108	0.383	0.000	0.000	141.1	42,396
Automatic runtime	0.033	0.992	0.950	0.060	43.9	13,148
Oracle ledger	0.758	0.983	0.942	1.000	39.9	5,907
LoRA block	0.008	0.075	0.658	0.065	55.9	20,457

operation recall is multiset overlap with supported gold operations. The prompts were frozen after a three-item plumbing check. In particular, the generic and automatic instructions retain a concrete 7×8 syntax example; demonstration anchoring observed in the full run was not tuned away. GSM8K has no matched no-compute controls or token-level alignment, so false-activation rate and first-wrong-token prevention are undefined.

10.2 Results

The deployable interfaces do not improve over LLM-only. Voluntary generic compute accepts calls on 114/120 rows, but 168 of 225 calls repeat the demonstration expression `7*8`; its duplicate-call and malformed-row rates are 92.5% and 5.0%, respectively, and registered-operation recall is zero. Automatic runtime accepts assistance on 114/120 rows but recalls only 6.0% of supported operations. It gains two and loses eleven pairs versus LLM-only. Among six rows where voluntary compute emits no valid call, automatic runtime rescues none, so the preregistered Automatic Rescue Rate is 0/6.

The base block prompt exactly ties LLM-only at 13/120 but swaps twelve gains for twelve losses and never opens a valid block. Its row is therefore a verbose elicitation baseline, not calculator evidence. LoRA opens a valid block on 79/120 rows, but 109/120 rows contain malformed block activity, operation recall is 6.5%, and only nine rows emit a scorable endpoint. It answers one row correctly. The same adapter's 16/16 synthetic held-out result therefore learned a narrow selection/serialization distribution rather than GSM8K decomposition.

The oracle ledger reaches 91/120 (75.8%, 95% Wilson interval 67.4–82.6%), with 79 paired gains and one loss versus LLM-only. This establishes large arithmetic-and-integration headroom when formalization is supplied, not a deployable routing result. Accuracy is not monotone in operation-count strata: oracle scores 60.0%, 87.5%, and 80.0%, so annotated operation count alone is not semantic difficulty.

This stress test changes the placement conclusion. Context or small adapter weights can store a stable local syntax contract, and the runtime can guarantee bounded exact execution, but none supplies the missing semantic decomposition on public word problems. Paper 1 therefore demonstrates synthetic interface feasibility and oracle public headroom, not calculator-assistance transfer.

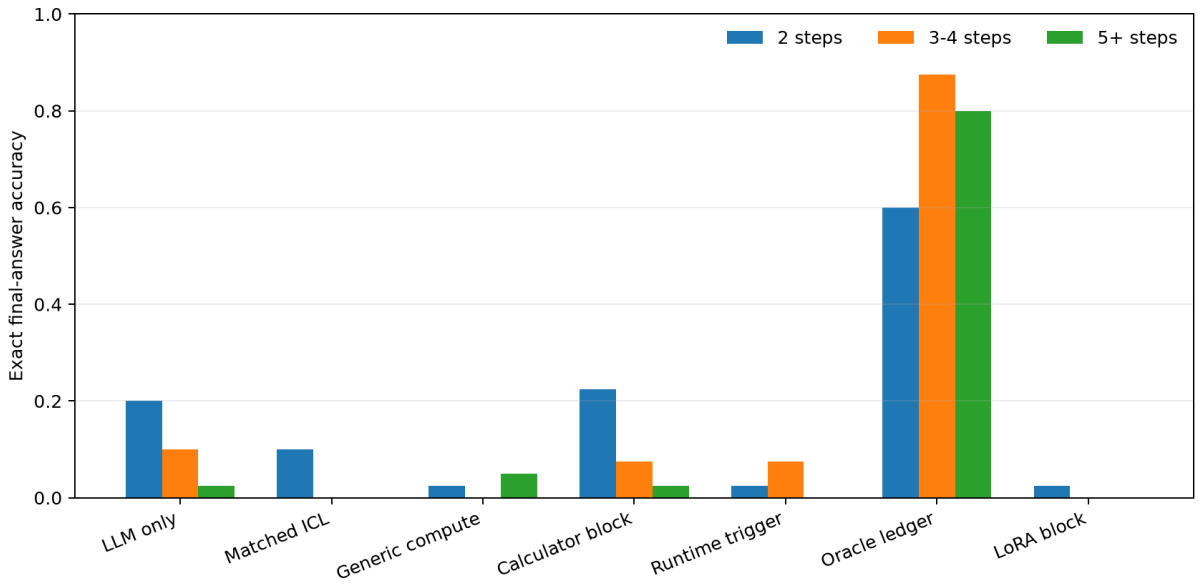

Figure 5: Exact GSM8K accuracy by registered operation-count stratum. Each bar contains 40 examples. Only the oracle ledger transfers strongly; deployable interfaces remain near zero across strata.

11 ASL-Arith Dataset Bootstrap

The public transfer failure localizes the next problem before calculator execution: compile linguistic relations into an executable representation. We therefore add the pipeline

NL clause → ASL-Arith → generic AST → typed CCIR → scoped workspace → operator registry.

ASL is the compact model-facing surface, for example `month2.downloads = month1.downloads * 3`; JSON/Python dictionaries are the canonical parsed representation. The parser knows syntax and precedence but not a closed function vocabulary. Semantic lowering and the active profile registry determine types and executors. Consequently future calls, predicates, variables, rules using `:-`, queries, and `SCOPE`/`END` already parse without claiming Paper 2 semantics in Paper 1.

Every dataset record owns one external root scope, such as `gsm8k:train:123`. Explicit ASL scopes augment that owner; they do not replace it. Unqualified reads search the current scope and permitted ancestors, while sibling reads require qualification. Unit tests cover arithmetic precedence, state updates, explicit nested scopes, qualified sibling references, future rule/retrieval syntax, and fail-closed dangling references. Arithmetic execution also supports causal forward dependencies: a relation may be deferred until a later clause grounds its inputs, after which dependents are reevaluated.

The TAT-QA adapter now carries table and paragraph evidence into each local teacher request; without this context, metric/year names and source values cannot be grounded from the question alone. The offset-preserving chop audit finds zero record/scope collisions and zero teacher-facing failures in arithmetic-compatible question clauses. Six GSM8K, ten CLUTRR, and three ProofWriter warnings remain in non-teacher rationale or ingestion fragments and stay in the review artifact. The audit therefore opens teacher bootstrap, not model training. Deterministic stratification freezes 100 GSM8K and 50 arithmetic TAT-QA examples, producing 343 and 50 potential

Table 9: Initial public DSL mining checkpoint. CLUTRR and ProofWriter are ingestion/extensibility records only and are not forced into ASL-Arith.

Dataset	Records	Arithmetic-compatible	Chopped parts
GSM8K train	7,473	7,473	52,705
TAT-QA development	1,644	706	2,477
CLUTRR test	1,146	0	7,761
ProofWriter test	360	0	7,056

clause-level teacher envelopes. Each envelope carries effective scope, current state, allowed operators, source identity, source evidence when available, and the compiler-skill bundle hash. Primary requests exclude benchmark answers, gold rationales, equations, and existing programs.

Before remote teaching, a deliberately lower-quality baseline converts only dataset-provided arithmetic annotations into ASL and verifies parse, scope, execution, and final answer. It accepts 95/100 GSM8K and 50/50 TAT-QA programs. Four GSM8K annotation chains do not terminate at the benchmark answer and one has no machine-readable equation; all five are quarantined. Accepted rows are graded `Q0_DATASET_DERIVATION_EXEC_VERIFIED`: numeric execution is gold-derived, variable names are generic steps, and the programs do not prove semantic NL-to-ASL compilation.

A stricter Codex skill then requires semantic paths, measured quantities, source-versus-derived state, temporal distinctions, unresolved dependencies, and named returns. Deterministic lint rejects anonymous operation-ledger targets and literal/expression-only returns. Thirty schema-constrained answer-blind Codex CLI batches yield 98/150 programs that pass syntax, CCIR lowering, scope, grounded execution, and hidden final-answer validation. The 52 rejects receive a labeled adjudication pass that exposes the dataset derivation as an arithmetic constraint: 45 pass after round one and seven after a second bounded repair, yielding 150/150. The final provenance split is therefore 98 primary, 45 repair-round one, and seven repair-round two. All rows remain `Q1_LOCAL_CODEX_EXEC_VERIFIED`; none is marked manual semantic gold.

The Click CLI can mine, audit, select, prepare answer-free Codex batches, prepare labeled repairs, validate semantic annotations, create the Q0 operation ledger, and call a configured LiteLLM teacher. Credentials remain provider-owned environment variables. The current semantic corpus used Codex CLI batches and made zero LiteLLM calls.

11.1 Frozen Q1 compiler pilot

We authorize the execution-verified Q1 corpus for a diagnostic pilot while retaining its non-manual quality label. A deterministic grouped split assigns 100 programs to training, 25 to development, and 25 to an untouched test set. Groups use renaming-invariant CCIR topology, semantic quantity family, and clause layout, so programs that differ only in entity names or constants cannot cross partitions. The frozen ledger has 92, 24, and 23 distinct pattern groups in train, development, and test, respectively, with zero pairwise overlap. Dataset counts are 66/34 GSM8K/TAT-QA in training and 17/8 in each held-out partition. The split was frozen before any model-facing run.

The experiment compares base Qwen3-0.6B, five- and ten-shot ASL prompting, rank-8 LoRA learning curves with nested 25-, 50-, and 100-program subsets, LoRA-100 plus three-shot prompting, and an augmentation adapter. The latter sees the 100 originals plus 900 deterministic, execution-verified train-only variants that rename grounded entities, remap safe numbers, and paraphrase question surfaces. Percentage parameters and calendar-year-like values are not inflated. No augmented descendant of a test or development parent enters training.

Table 10: Untouched 25-program Q1 test. Parse and lower are validity rates; Exec. is deterministic ASL execution; Answer is final executed-answer accuracy. Exact ASL is 0 in every condition. Tokens are mean prompt tokens.

Condition	Parse	Lower	Exec.	Answer	Tokens
Base	.00	.00	.00	.00	234
Five-shot ICL	.76	.76	.64	.12	2,225
Ten-shot ICL	.88	.88	.76	.16	3,758
LoRA-25	.88	.72	.48	.20	234
LoRA-50	.92	.88	.68	.28	234
LoRA-100	.92	.84	.68	.28	234
LoRA-100 + 3-shot ICL	.88	.84	.72	.40	1,497
LoRA-B augmented	.96	.92	.64	.24	234
LoRA-500	.88	.84	.76	.44	234

Evaluation separates the 25 untouched semantic programs from numeric remapping, very-large-number remapping, and paraphrase suites. Twenty of 25 test programs contain a safely surface-matched number eligible for each numeric suite; all 25 are eligible for paraphrase. Exact ASL is retained but is not the primary outcome: we also score parse and semantic-lint validity, entity/path grounding, source facts, operators, references and dependency edges, execution, canonical return and state equivalence under harmless path renaming, and final executed answer. With only 25 untouched programs, this pilot can reveal a learning signal and the kind of invariance learned by augmentation; it cannot establish a robust general semantic compiler.

11.2 Q1 compiler-pilot results

The 100 training programs contain 92 normalized semantic-pattern signatures: 87 occur once, three twice, one three times, and one four times. Thus the corpus is structurally diverse under the registered signature, although that signature cannot prove complete semantic diversity. Relation tags expose remaining thin classes, including only six percentage and 16 relative-quantity programs, compared with 45 ratio/fraction and 55 remaining/difference programs. Future selection therefore targets relation-class deficits rather than uniform rows.

Zero-shot Qwen3-0.6B emits natural-language summaries and no parseable ASL. Demonstrations establish the surface contract immediately but cost 9.5–16 times the base prompt tokens. LoRA-25 already reaches 5/25 answers without this prompt overhead; LoRA-50 rises to 7/25 and improves path, source-fact, operator, and state-component F1. LoRA-100 remains at 7/25 and regresses on several component scores despite a lower development loss. The curve therefore plateaus within this small test rather than supporting monotonic scaling. Three residual demonstrations complement LoRA-100, reaching 10/25; the wide Wilson interval (.234, .593) and single seed preclude a strong placement claim.

The same QKVO rank-8 adapter trained on the relation-diversified 450-row training set reaches 11/25 answers (44%; 95% Wilson interval .267–.629), with 22/25 parseable, 21/25 lowerable and type-valid, and 19/25 executable programs. This is four more answers than LoRA-100 and one more than LoRA-100 plus three-shot ICL, without demonstration tokens. Dataset breakdown is asymmetric: TAT-QA reaches 7/8 answers while GSM8K reaches 4/17. Thus added semantic originals produce a whole-program learning gain, but much of it may reflect TAT-QA specialization rather than broad GSM8K transfer.

All adapters modify 2,293,760 QKVO parameters (0.383% of the model). LoRA-25, -50, and

Table 11: LoRA-A (100 originals) versus LoRA-B (100 originals plus 900 train-only perturbations). Numeric suites contain the 20 eligible test parents; original and paraphrase contain all 25.

Suite	LoRA-A			LoRA-B		
	Lower	Exec.	Answer	Lower	Exec.	Answer
Original	.84	.68	.28	.92	.64	.24
Numeric	.75	.55	.25	.90	.65	.25
Very large	.80	.50	.15	.90	.60	.20
Paraphrase	.80	.64	.28	.92	.64	.32

-100 consume 29,550, 59,640, and 112,950 target tokens and finish in 427, 836, and 1,462 seconds on XPU. Their final development losses are .682, .642, and .579. LoRA-B consumes 235,470 target tokens in 3,176 seconds and reaches .535 development loss, so its comparison is not compute matched. LoRA-500 retains the same 2,293,760 trainable parameters and ten-epoch schedule, consumes 565,160 target tokens, takes 6,832 seconds, and reaches .507 development loss. It changes data coverage and total token exposure together; it is not a compute-matched ablation against LoRA-100.

Augmentation does not improve untouched semantic-program answers (6/25 versus 7/25), but it consistently improves lowerability and improves very-large-number answers from 3/20 to 4/20. Relative to its own original outputs, LoRA-B loses no correct parent under ordinary numeric remapping or paraphrase, whereas LoRA-A loses two and one, respectively. The evidence therefore supports a narrow interpretation: controlled perturbations teach surface and magnitude invariance, not new reasoning structures. Parsing, CCIR lowering, semantic type validity, and execution are recorded separately. In particular, one parseable Boolean arithmetic output lowers mechanically but is type-invalid and cannot execute; it receives no state/execution correctness.

11.3 Relation-diversified 500-original checkpoint

We next freeze the data checkpoint that precedes any larger-adapter comparison. Selection excludes all 150 prior source IDs and greedily balances answer-free surface cues for aggregation, comparison, equal allocation, multiple entities, percentages, rates, ratios/fractions, relative quantities, remaining/difference, temporal shifts, chains, and nested dependencies. We draw a 400-row reserve (267 GSM8K and 133 TAT-QA), rather than another uniform sample. Eighty answer-blind Codex batches produce 267 programs that pass semantic lint, parse, CCIR lowering and types, grounded execution, and answer agreement. One labeled answer-visible repair round recovers 124 of 133 rejects, for 391/400 accepted programs: 267 answer-blind primary and 124 rationale-assisted repair. Nine rows remain rejected.

The post-compilation freeze recomputes each normalized semantic signature and excludes the 47 frozen development/test pattern families. Seven otherwise accepted rows collide and are quarantined. From 384 eligible programs we choose exactly 350, retaining 34 as reserve. The selected expansion contains 260 GSM8K and 90 TAT-QA programs and 314 distinct signatures; 303 signatures occur once. Combined with the original frozen 100-row training partition, the 450 training programs contain 397 signatures, with only nine signatures shared between old and new training data. Frozen development and test remain unchanged at 25 rows each, giving 500 semantic originals total and zero expansion overlap with either held-out pattern set.

The expansion campaign reports 1,320,932 primary and 642,753 repair tokens (1,963,685 total) and 1,697 plus 565 seconds of wall time under four concurrent local Codex workers. These are teacher-generation costs, not student-training tokens. Relation labels are multi-valued: among the

Table 12: Frozen 25-program whole versus incremental comparison. Pred. is the strict deployed predicted-state loop; Oracle supplies only gold prior state. Intervals are 95% Wilson intervals for final-answer accuracy.

Condition	Parse	Lower	Exec.	State	Answer	Answer interval
Whole program	.88	.84	.76	.20	11/25	[.267,.629]
Incremental pred.	.92	.92	.60	.16	7/25	[.143,.476]
Incremental oracle	.92	.88	.28	.16	6/25	[.115,.434]
Pred. + full question	.88	.88	.64	.20	11/25	[.267,.629]

selected 350, for example, 171 carry ratio/fraction cues, 130 relative-quantity cues, 138 percentage cues, 175 aggregation cues, and 232 chained-relation cues. Labels describe selection coverage, not independently adjudicated semantic classes. This checkpoint alone makes no student-model improvement claim. The subsequent same-rank student run, reported above, improves untouched answers from 7/25 to 11/25. We next derive 1,403 causal clause-local training transitions from the 450 programs, with only the current clause, compact evidence, scope, and prior executed state visible; the 65-transition development set retains the original 25 development parents. That incremental-state condition is the fixed input to the rank comparison below.

11.4 Incremental executed-state compiler

The clause-local condition uses the same pinned Qwen3-0.6B revision and rank-8 QKVO placement as the whole-program adapter. Each training row contains only the current natural-language clause, compact evidence and scope, and the workspace obtained by executing verified prior ASL; answers and future clauses are hidden. The 450 training programs yield 1,403 transitions from 397 normalized semantic signatures, and the unchanged 25-program development set yields 65 transitions. Ten epochs consume 576,570 target tokens in 1,760 optimizer steps. Training takes 25,762 seconds (7.16 hours), peaks at 7.24 GiB XPU memory, and ends at train/dev loss .029/1.082. The adapter has 2,293,760 trainable parameters and its stored weight hash begins `dc7191b9`; the full provenance is machine-readable.

At test time, the deployed condition repeatedly predicts one ASL delta, validates syntax, lowering, and type safety, executes an accepted delta, and feeds the resulting workspace to the next clause. It stops at the first invalid delta and does not repair. A diagnostic oracle-state condition instead supplies the gold executed state before each clause, independently of prior student output, while leaving the current prediction untouched. Both use the same frozen 25 source IDs, model checkpoint, adapter, decoding seed, and metric definitions.

The primary predicted-state loop attempts 63 of 70 reference transitions before three fail-closed stops, accepts 60/63 (95.2%), completes all clauses in 22/25 programs, and has mean completed fraction .888. Delta parse, lower, type, and execution rates are .968, .952, .952, and .762. Yet exact operator, semantic path, source-fact, dependency, and state-delta rates are only .254, .175, .397, .175, and .365. Thus the adapter usually speaks valid ASL but often encodes the wrong local relation. Oracle state attempts all 70 transitions and accepts 65; its corresponding semantic exact rates remain .257, .300, .414, .186, and .343.

Final-answer accuracy falls from 11/25 whole-program to 7/25 incremental. Paired outcomes are seven both correct, 14 both wrong, zero whole-wrong/incremental-correct, and four whole-correct/incremental-wrong (two-sided exact binomial $p = .125$, exploratory). Oracle state reaches 6/25, a propagation gap of $-.04$: it recovers no predicted-state answer and loses one. This single-seed diagnostic therefore does not support the incremental-state-advantage hypothesis and gives

Table 13: Matched incremental adapter-rank comparison on the frozen 25 programs. All entries except Answer are program-level rates. Full adds the untouched original question to each predicted-state prompt.

Rank	Context/state	Parse	Exec.	State	Dependency	Answer
8	Causal predicted	.92	.60	.16	.24	7/25
16	Causal predicted	1.00	.68	.12	.12	7/25
8	Causal oracle	.92	.28	.16	.20	6/25
16	Causal oracle	.92	.32	.12	.12	6/25
8	Full predicted	.88	.64	.20	.20	11/25
16	Full predicted	.84	.64	.12	.16	10/25

no evidence that early workspace errors are the primary limitation.

The prespecified context-loss control keeps predicted-state execution but adds the untouched full original question to every local prompt. It reaches 11/25, gaining five causal-loop failures and losing one causal-loop success (two-sided exact binomial $p = .219$, exploratory). Against whole-program LoRA it has eight both correct, 11 both wrong, three gains, and three losses ($p = 1.0$). The control therefore recovers the causal chopping penalty but does not establish an incremental executed-state advantage. Parse/lower/execution are .88/.88/.64; GSM8K rises from 1/17 to 5/17 while TAT-QA remains 6/8.

The failure is sharply dataset-dependent. Predicted-state incremental retains 6/8 TAT-QA answers but only 1/17 GSM8K answers; oracle state retains 6/8 and 0/17. The frozen GSM8K programs average 3.65 clauses, 7.65 ASL statements, dependency depth 3.47, 2.06 entities, and 6.59 paths, versus 1.00 clause, 4.50 statements, depth 2.13, 1.50 entities, and 3.50 paths for TAT-QA. Among attempted GSM8K deltas, operator, path, and dependency exactness are .182, .182, and .109. The multi-label error audit likewise concentrates wrong paths, dependencies, operators, and returns in GSM8K. Because TAT-QA is effectively one-part in this freeze while GSM8K is genuinely sequential, this asymmetry is consistent with a local grounding and global-context bottleneck; it is not proof of a dataset-level causal explanation. The full-question control confirms that global intent matters, but its tie with whole-program synthesis leaves local semantics unresolved. The matched rank ablation below isolates adapter capacity without mistaking context removal for an architectural gain.

11.5 Adapter rank and non-data interventions

We doubled QKVO rank from 8 to 16 while holding the 1,403 training transitions, 65 development transitions, ten epochs, base-model revision, seed, decoding, and frozen test programs fixed. Rank 16 has 4,587,520 trainable parameters (0.764% of the resulting model), trains in 14,248 seconds (3.96 hours), peaks at 7.27 GiB XPU memory, and ends at train/dev loss .022/1.167. The matched rank-8 losses are .029/1.082. Thus the larger adapter fits training transitions more tightly but generalizes worse by development loss.

Doubling rank changes neither causal predicted-state nor oracle-state answers. The causal paired comparison has six both correct, 17 both wrong, one rank-8-only success, and one rank-16-only success ($p = 1.0$, exploratory). Rank 16 removes the two rank-8 program-level parse failures and raises execution from .60 to .68, but semantic state falls from .16 to .12, dependency correctness from .24 to .12, and transition operator exactness from .254 to .188. In the full-question control it reaches 10/25 rather than 11/25, with two gains and three losses ($p = 1.0$). This diagnostic does not support insufficient QKVO rank as the current bottleneck.

The state and context ablations distinguish two kinds of information. The executed workspace is mostly *extensional*: it records which paths currently have which values. The original question is *intensional*: it preserves entity roles, relation provenance, category meaning, the requested target, and why a derived value should be computed. Supplying gold extensional state changes answers from 7/25 to 6/25 at both ranks; it recovers some individual path or state deltas but no net final answers. Supplying the full narrative while keeping predicted state instead changes rank-8 from 7/25 to 11/25 and rank-16 from 7/25 to 10/25. For example, in the frozen necklace case, both causal variants treat turquoise as a literal third type/count. Full context restores the residual relation and computes $40 - 7 - 14 = 19$.

This is not evidence that full context repairs workspace integration. Rank-8 full-context path exactness is .097 and rank-16 is .086; their semantic-state rates are only .20 and .12. A model can therefore recover the answer by recomputing from the question while emitting duplicated or semantically poor state. The supported conclusion is narrower: removing global intent is more damaging than accumulated state error under this representation.

The deterministic intervention analysis consequently ranks retaining the raw observed question or a model-generated intent ledger first. A gold semantic frame is unavailable at test time and is not an admissible input. Factorized path selection may use only symbols already created by accepted model output; the model must still propose unseen targets from raw text. Epoch-level development evaluation, early stopping, and regularization are also warranted by the .029/.022 training losses versus 1.082/1.167 development losses. A fixed-demonstration hybrid is supported by the earlier LoRA-100 plus three-shot gain of .12: one seed-fixed training set was used for every test item, with no per-problem retrieval. QKVO+MLP at rank 8 remains the clean adapter-location ablation after the rank plateau, and a 1–1.7B base-model replication can test base capacity. Grammar-constrained decoding is useful for fail-closed safety, but rank 16 already converts an eight-point parse gain into zero answer gain. These interventions may complement additional distinct semantic programs; none yet substitutes for an independently evaluated larger corpus.

11.6 Fine-grained semantic failure decomposition

We replayed the three frozen prediction files through a deterministic AST/CCIR diagnostic rather than generating new model output. The analysis separates surface validity and executed answers from entity, attribute, qualifier, relation, source-fact, dependency, and query grounding. Whole-program symbol alignment permits only one-to-one, lexically compatible renames with matching graph roles; incremental symbols are sticky once present in the workspace. Metrics below are therefore strict deterministic values. Twenty-six condition–program records with plausible but unresolved symbol equivalence are placed in an optional judge queue and receive no automatic credit. Generic ASL type validity does not establish dimensional compatibility, which remains explicitly not measurable under the current arithmetic type system.

The decomposition supports the prespecified broad claim but materially sharpens it. Whole-program operator mapping is stronger than strict attributes, source-fact attachment, dependencies, relation direction, aggregation, temporal qualifiers, and the returned semantic slot. In particular, only 12/30 supported directed relations and 7/25 RETURN targets are correct. Correct arithmetic can mask this weakness: all five semantically state-equivalent programs answer correctly, but 6/20 state-inequivalent programs also reach the benchmark answer. Among parse/type-valid programs, 9/21 have exact operator multisets while only 2/21 satisfy the bounded full semantic-name/graph equivalence test. All seven operator-perfect but path-incorrect cases nevertheless return the right answer. Thus answer agreement is not evidence of a reusable semantic workspace.

Incremental localization modestly raises entity, attribute, qualifier, and operator scores but does

Table 14: Fine-grained whole-program LoRA-500 semantics on the frozen 25-program test. F1 is micro-averaged over the listed reference support; accuracy is correct/support for deterministically comparable relations.

Component	Score	Support	Component	Score	Support
Entity F1	.538	47	Attribute F1	.144	140
Qualifier F1	.387	67	Operator F1	.695	94
Argument order	.429	7	Relation direction	.400	30
Source-fact F1	.227	85	Dependency F1	.113	121
Aggregation/cardinality	.080	25	Temporal grounding	.300	10
RETURN target	.280	25			

Table 15: Fine-grained program semantics by frozen condition. Reuse is exact reuse of an existing gold semantic path over 60 transition references and is undefined for standalone whole-program synthesis. Other columns are program-level micro-F1 except direction and RETURN, which are support-aware accuracies.

Condition	Entity	Attribute	Qualifier	Operator	Direction	Dependency	RETURN	Reuse
Whole LoRA-500	.538	.144	.387	.695	.400	.113	.280	–
Incremental pred.	.619	.201	.480	.762	.381	.125	.240	.033
Incremental oracle	.629	.435	.654	.743	.455	.139	.320	.450

not improve final answers, direction, dependencies, or RETURN grounding. More importantly, the deployed predicted-state loop exactly reuses only 2/60 existing semantic references, versus 27/60 under oracle prior state. This identifies symbol continuity as a concrete stateful failure, while the oracle condition's 6/25 answer result shows that improved naming continuity alone is insufficient. The strict attribute and dependency scores are the weakest measured whole-program components, but unresolved ontology aliases and the 25-program sample prevent claiming a universal primary cause. The safe artifacts retain dataset, semantic-family, complexity, conditional, teacher-name consistency, ID-only error examples, and input/output hashes; full records remain local under repository benchmark-text policy.

12 Preliminary XPU Pilot

12.1 Setup

We ran a diagnostic pilot with Qwen3-0.6B at pinned revision `c1899de` (the full hash is stored in the manifest), PyTorch 2.13.0+XPU, and an Intel integrated GPU. The deterministic sample crossed one and three operators with one- and two-digit operands, using addition and multiplication. It contained two examples per cell (eight arithmetic examples) and eight matched non-trigger controls. One generation seed (17) and five conditions yielded 80 generations. The 96-token output budget was selected after a shorter plumbing run exposed truncation; the shorter run is not included in the results. Confidence intervals are Wilson intervals over only eight arithmetic observations per condition and are therefore intentionally wide.

Table 16: Exploratory XPU pilot. Accuracy intervals are 95% Wilson intervals; tokens and latency are means. Latency reflects the correctness-first full-prefix adapter and is not a production estimate.

Condition	Accuracy	Trigger recall	Output tokens	Latency (ms)
LLM only	0.750 [0.409, 0.929]	—	40.56	8028
Matched prompt	0.625 [0.306, 0.863]	—	45.81	9380
Explicit tool	0.875 [0.529, 0.978]	0.625	22.50	4410
Reflex	0.750 [0.409, 0.929]	0.750	46.06	9195
Oracle	1.000 [0.676, 1.000]	1.000	15.31	2718

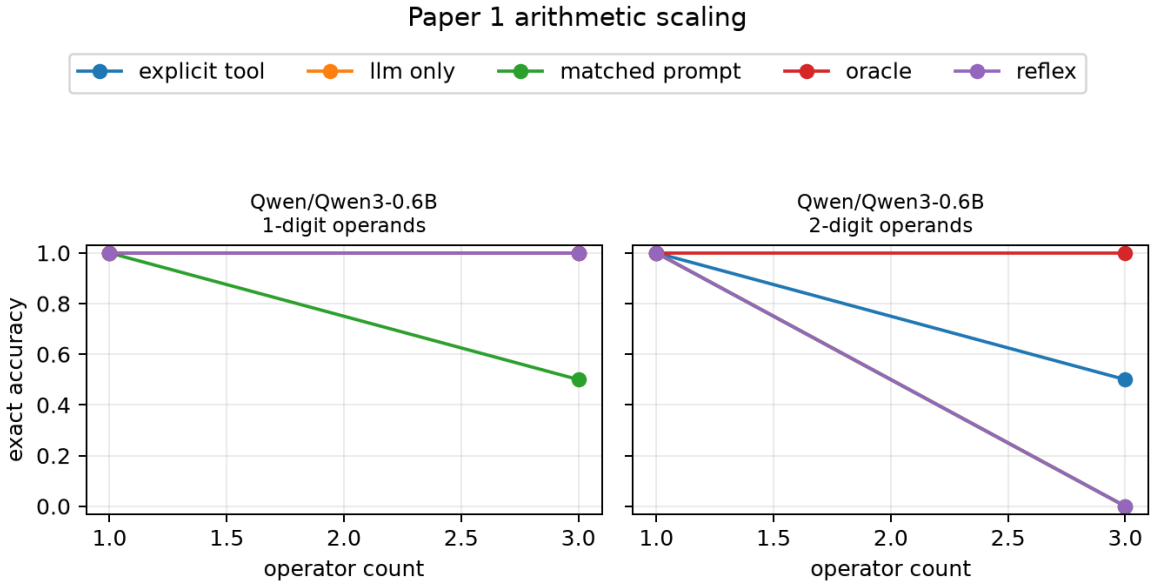

Figure 6: Exploratory exact-match accuracy by generated difficulty cell. Each point contains only two examples; the figure diagnoses plumbing and candidate behavior and must not be read as a scaling estimate.

12.2 Results

The reflex condition did not improve exact-match accuracy over LLM only in this pilot. Explicit-tool accuracy was numerically higher and oracle intervention was perfect, but the sample is too small for either difference to support a general claim. No condition falsely intervened on a control. Accepted explicit-tool candidates represented the intended expression in every triggered case, whereas only 50% of reflex candidates did. Inspection showed that the strict watcher often intercepted a valid intermediate subexpression rather than the full task expression. Once a candidate reached the engine, execution correctness was 100% in both conditions. Reflex result-use was 66.7%, and one third of used results were later overridden. These component results locate the present bottleneck in exposure, selection, and downstream use rather than arithmetic execution.

At this protocol revision H1 is not supported: the automatic reflex tied the unassisted baseline and failed to dominate explicit invocation. H2–H4 remain untested because this pilot includes neither a production interface nor the required model-size and seed sweeps. Paper 2 therefore remains gated. A confirmatory run must increase examples per cell, use all declared seeds and model sizes, add a second family, and freeze any detector change before examining outcomes.

13 Falsification and Evidence Gate

The reflex claim is not supported if any of the following holds under matched capability and declared cost accounting:

- answer gains disappear when final model claims, rather than inserted engine values, are scored;
- false triggers, normalization errors, or ignored results offset exact execution gains;
- explicit calculator calling matches or dominates the quality–token–latency frontier;
- reflex and baseline accuracy have indistinguishable difficulty slopes;
- gains occur only with oracle detection or only in one easy/model cell; or
- results fail to reproduce across seeds or a second model family.

Paper 2 is justified only if Paper 1 identifies a reproducible regime where automatic strict assistance helps and mixed operation types expose a limitation of one calculator. Paper 3 is not justified by syntax brittleness alone; learned interrupts require a measured task regime worth extending.

14 Reproduction

The reusable runtime lives under `src/ccpu/common`; Paper 1 code lives under `src/ccpu/paper1`. From the repository root, the dependency-free protocol check is:

```
python -m pip install -e .
python -m pytest
python -m ccpu paper1 generate \
  --config configs/paper1/smoke.json \
  --output artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 validate \
  --dataset artifacts/paper1/smoke/dataset.jsonl
python -m ccpu paper1 simulate \
  --dataset artifacts/paper1/smoke/dataset.jsonl \
  --output-dir artifacts/paper1/smoke/run
```

The pinned pretrained path is:

```
python -m pip install -e ".[hf]"
python -m ccpu paper1 generate \
  --config configs/paper1/primary.json \
  --output artifacts/paper1/primary/dataset.jsonl
python -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/primary/dataset.jsonl \
  --config configs/paper1/primary.json \
  --output-dir artifacts/paper1/primary/run
```

The reported XPU pilot uses the direct interpreter from the validated XPU environment and can be reproduced with:

```
$py = 'C:\Users\j.simao\.venvs\modal-llm-xpu\Scripts\python.exe'
$env:PYTHONPATH = (Resolve-Path src).Path
& $py -m ccpu paper1 generate \
  --config configs/paper1/xpu_pilot.json \
  --output artifacts/paper1/xpu_pilot/dataset.jsonl
& $py -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/xpu_pilot/dataset.jsonl \
  --config configs/paper1/xpu_pilot.json \
  --output-dir artifacts/paper1/xpu_pilot/run96
& $py -m ccpu paper1 replay \
  --dataset artifacts/paper1/xpu_pilot/dataset.jsonl \
  --completions artifacts/paper1/xpu_pilot/run96/predictions.jsonl \
  --output-dir artifacts/paper1/xpu_pilot/final
```

The held-out comparison and derived analysis use:

```
& $py -m ccpu paper1 run-hf \
  --dataset artifacts/paper1/hard_heldout_xpu/dataset.jsonl \
  --config configs/paper1/hard_heldout_xpu.json \
  --output-dir artifacts/paper1/hard_heldout_xpu/run \
  --device xpu
& $py -m ccpu paper1 paired \
  --dataset artifacts/paper1/hard_heldout_xpu/dataset.jsonl \
  --predictions artifacts/paper1/hard_heldout_xpu/run/predictions.jsonl \
  --output artifacts/paper1/hard_heldout_xpu/run/paired_analysis.json
python -m ccpu paper1 plot-interfaces \
  --summary artifacts/paper1/hard_heldout_xpu/run/summary.json \
  --output docs/papers/paper1/paper1_heldout_interfaces.png
```

Device selection for `device: auto` is CUDA, then XPU, then CPU. The replay step applies the current frozen evaluator to preserved generations and records both the source prediction hash and rescore provenance.

Use `--limit`, `--model`, and repeated `--condition` arguments for resource preflight. Each run writes predictions, full traces, summary tables, hashes, configuration, model metadata, environment information, and git revision/dirty status. The replay command permits model generation in a separate harness while preserving identical controller and evaluation semantics. The `plot` command renders the machine-readable scaling cells as accuracy curves by model, operand width, condition, and operator count. The `compare-models` command consumes the frozen comparison configuration and regenerates the cross-model table plus block-capability, interface-accuracy, token–latency, and failure-mode figures without manually entered results. The `generate-lora-data` and `train-lora` commands implement the versioned protocol-only adapter phase; the former must pass its leakage audit before the latter can run. The `analyze-placement` command hashes the source summaries and training reports and regenerates Table 7 and Figure 4 inputs.

The public transfer run is resumable by condition and is reproduced with:

```
& $py -m ccpu paper1 freeze-public-gsm8k \
  --source-selection artifacts/paper2/public_benchmarks/data_v1/selection.jsonl \
  --output-dir artifacts/paper1/public_gsm8k_v1/data
foreach ($condition in @('llm_only','matched_icl','generic_compute', \
```

```
  'calculator_block','runtime_trigger','oracle_calculator', \
  'lora_calculator_block')) {
  & $py -m ccpu paper1 run-public-gsm8k \
    --public-config configs/common/public_benchmarks.json \
    --cache-root C:\Users\j.simao\.cache\ccpu-public-v1 \
    --selection artifacts/paper1/public_gsm8k_v1/data/selection.jsonl \
    --model-config configs/paper1/public_gsm8k_qwen_xpu.json \
    --condition $condition --checkpoint-every 10 \
    --output-dir artifacts/paper1/public_gsm8k_v1/qwen3_0_6b/$condition
}
$analysis = @('paper1','analyze-public-gsm8k','--output-dir', \
  'artifacts/paper1/public_gsm8k_v1/analysis')
foreach ($condition in @('llm_only','matched_icl','generic_compute', \
  'calculator_block','runtime_trigger','oracle_calculator', \
  'lora_calculator_block')) {
  $analysis += @('--predictions', \
    "artifacts/paper1/public_gsm8k_v1/qwen3_0_6b/$condition/predictions.jsonl")
}
& $py -m ccpu @analysis
```

The ASL dataset bootstrap uses verified local public-source files:

```
DSL=artifacts/paper1/dsl
python -m ccpu.dsl_dataset mine \
  --source gsm8k=<pinned-gsm8k-train.parquet> \
  --source tatqa=<pinned-tatqa-development.json> \
  --source clutrr=<pinned-clutrr.parquet> \
  --source proofwriter=<pinned-proofwriter.parquet> \
  --source-split tatqa=development \
  --source-split clutrr=test \
  --source-split proofwriter=test \
  --output-dir artifacts/paper1/dsl/raw_v1
python -m ccpu.dsl_dataset audit-chops \
  --input-dir artifacts/paper1/dsl/raw_v1 \
  --output artifacts/paper1/dsl/raw_v1/chop_audit.json
python -m ccpu.dsl_dataset select \
  --input artifacts/paper1/dsl/raw_v1/gsm8k.jsonl \
  --max-examples 100 \
  --output artifacts/paper1/dsl/bootstrap_v1/gsm8k_seed.jsonl
python -m ccpu.dsl_dataset bootstrap-annotated \
  --input artifacts/paper1/dsl/bootstrap_v1/gsm8k_seed.jsonl \
  --input artifacts/paper1/dsl/bootstrap_v1/tatqa_seed.jsonl \
  --output-dir artifacts/paper1/dsl/operation_ledger_v1
python -m ccpu.dsl_dataset prepare-local \
  --seed artifacts/paper1/dsl/bootstrap_v1/gsm8k_seed.jsonl \
  --seed artifacts/paper1/dsl/bootstrap_v1/tatqa_seed.jsonl \
  --output-dir artifacts/paper1/dsl/local_codex_v1
python -m ccpu.dsl_dataset validate-semantic \
```

```
  --seed artifacts/paper1/dsl/bootstrap_v1/gsm8k_seed.jsonl \
  --seed artifacts/paper1/dsl/bootstrap_v1/tatqa_seed.jsonl \
  --annotations ${DSL}/local_codex_v1/annotations_primary.jsonl \
  --annotations ${DSL}/local_codex_repair_v1/annotations_repair1.jsonl \
  --annotations ${DSL}/local_codex_repair_v2/annotations_repair2.jsonl \
  --output-dir artifacts/paper1/dsl/annotated_v1
python -m ccpu.dsl_dataset generate --dry-run \
  --input artifacts/paper1/dsl/bootstrap_v1/gsm8k_seed.jsonl \
  --config configs/paper1/dsl_teacher.example.json \
  --skill skills/ccir-arith-compiler/SKILL.md \
  --output-dir artifacts/paper1/dsl/teacher_dry_run

# Relation-diversified 500-original checkpoint
python -m ccpu.dsl_dataset select-diverse \
  --input ${DSL}/raw_v1/gsm8k.jsonl \
  --input ${DSL}/raw_v1/tatqa.jsonl \
  --exclude ${DSL}/annotated_v1/accepted.jsonl \
  --dataset-target gsm8k=267 --dataset-target tatqa=133 \
  --output ${DSL}/expansion500_v1/candidates.jsonl
python -m ccpu.dsl_dataset prepare-local \
  --seed ${DSL}/expansion500_v1/candidates.jsonl \
  --output-dir ${DSL}/expansion500_v1/primary
python -m ccpu.dsl_dataset run-local \
  --requests-dir ${DSL}/expansion500_v1/primary/requests \
  --output-dir ${DSL}/expansion500_v1/primary \
  --prompt configs/paper1/local_codex_annotation_prompt.md \
  --schema configs/paper1/semantic_annotation_batch.schema.json
python -m ccpu.dsl_dataset validate-semantic \
  --seed ${DSL}/expansion500_v1/candidates.jsonl \
  --annotations ${DSL}/expansion500_v1/primary/annotations.jsonl \
  --output-dir ${DSL}/expansion500_v1/validated_primary
python -m ccpu.dsl_dataset prepare-repair \
  --seed ${DSL}/expansion500_v1/candidates.jsonl \
  --rejected ${DSL}/expansion500_v1/validated_primary/rejected.jsonl \
  --output-dir ${DSL}/expansion500_v1/repair1 --repair-round 1
python -m ccpu.dsl_dataset run-local \
  --requests-dir ${DSL}/expansion500_v1/repair1/requests \
  --output-dir ${DSL}/expansion500_v1/repair1 \
  --prompt configs/paper1/local_codex_repair_prompt.md \
  --schema configs/paper1/semantic_annotation_batch.schema.json
python -m ccpu.dsl_dataset validate-semantic \
  --seed ${DSL}/expansion500_v1/candidates.jsonl \
  --annotations ${DSL}/expansion500_v1/primary/annotations.jsonl \
  --annotations ${DSL}/expansion500_v1/repair1/annotations.jsonl \
  --output-dir ${DSL}/expansion500_v1/validated_repair1
python -m ccpu.dsl_dataset finalize-expansion \
  --accepted ${DSL}/expansion500_v1/validated_repair1/accepted.jsonl \
  --existing ${DSL}/annotated_v1/accepted.jsonl \
```

```
--frozen-ledger artifacts/paper1/asl_pilot_v1/freeze/split_ledger.jsonl \
--output-dir ${DSL}/expansion500_v1/freeze --target 350
```

15 Limitations

The elicitation instruction asks reflex models to expose an expression followed by an equals sign. This is less explicit than an API call but still changes generation behavior, and a model that answers directly may never trigger. Both trigger rate and conditional-on-trigger accuracy are therefore necessary. The strict lexical grammar does not understand intent and will miss prose arithmetic; expanding semantics belongs to Paper 3. Quote suppression is deliberately limited, and code-like or unusual formatting remains an adversarial surface.

The synthetic task controls arithmetic complexity but not realistic discourse. The textual explicit-tool baseline may understate a model's native structured function-calling ability. Full-prefix token recomputation overstates runtime cost. The main comparisons have one greedy seed, 16 arithmetic examples, and four observations per selected cell. Their intervals remain wide, the adaptive bands are coarse, and no architecture or universal scaling generalization is supported. The 4B row is only a six-item smoke. Llama remains unavailable behind manual authorization; approved Gemma3-1B and ungated SmolLM2 provide two cross-family adapter replications but cannot establish broad family effects. The adapter data are synthetic and small, and near-zero development loss may reflect the narrow protocol distribution. Twelve controls do not tightly bound rare false triggers, as the ICL failures illustrate. Adapter+minimal was selected by the predeclared gate, but the three-family comparison remains developmental rather than confirmatory. The zero-shot block prompt's historical result is not reinterpreted by the separately developed ICL prompt, endpoint audit, or LoRA result. Finally, exact calculator output cannot repair an incorrectly copied expression, a model that ignores the result, or a task whose premises are wrong. The public transfer slice adds further limits: it has one small checkpoint, one greedy seed, and no no-compute controls. Operation-count strata are not a complete difficulty model. Source rationales provide noisy opportunity annotations, and 10.1% of annotated operations fall outside the registered calculator surface. The concrete syntax example causes strong anchoring in generic and automatic conditions; because it was frozen before the full outcome, this is valid evidence for that protocol but not evidence that all tool or trigger prompts must fail. The ASL corpus is suitable only for the explicitly labeled Q1 pilot. Although all 500 frozen semantic programs execute to the benchmark answer, 176 required rationale-assisted repair (52 in the initial checkpoint and 124 in the expansion), and deterministic lint cannot establish that every entity, quantity, or temporal field is the best semantic representation. No row is Q4 manual gold. The grouped split prevents measured topology leakage but does not replace independent-teacher agreement or manual semantic adjudication. Twenty-five untouched test programs also give imprecise whole-program estimates; the richer component metrics are diagnostic rather than extra independent trials. The 900 perturbations share only 100 semantic parents and LoRA-B has roughly twice LoRA-100's target-token exposure, so robustness differences cannot be attributed to augmentation alone. The normalized pattern signature is useful for leakage control but may merge or separate structures imperfectly. The 500-original checkpoint adds 350 relation-diversified training programs while excluding every frozen development/test semantic-pattern family. Its QKVO-r8 result changes semantic coverage and total token exposure together. Incremental NL+executed-state training and the matched rank-16 ablation show that doubling QKVO rank does not move held-out answers. QKVO+MLP placement, context preservation, epoch selection, and base-model scale remain developmental tests.

16 Related Work

Toolformer trains language models to decide when and how to invoke APIs, including a calculator [1]. PAL delegates model-generated programs to an interpreter [2], while ReAct interleaves model-authored reasoning and actions [3]. These systems establish the value of external execution. Paper 1 isolates a different control point: a runtime recognizes ordinary strict arithmetic already emerging in the output and invokes a bounded engine without a function-name decision. The comparison to explicit invocation is empirical; the paper does not assume that the reflex interface is better.

LoRA freezes base weights and injects trainable low-rank matrices [4]; our use of it is conventional, and the empirical question is only whether a small adapter can store this interface contract. Self-RAG-like systems show that models can learn control or reflection tokens for inference actions [5]. Calculator fences are a typed local execution region, not a claim to have invented trained action syntax or control tokens.

17 Conclusion

Paper 1 reduces the cognitive-coprocessor proposal to its smallest credible experiment. A character-incremental detector, typed allowlisted micro-IR, exact bounded calculator, append-only state, and immediate text reinjection are enough to test whether automatic computation can improve an autoregressive stream. The original strict watcher does not improve accuracy, and the zero-shot fenced block usually emits invalid content. Deterministic surface normalization is numerically stronger, executing 12 of 16 full expressions and improving five paired examples without a loss, but remains an unreplicated result. Separate development shows that the block failure was not intrinsic to the grammar: one frozen two-shot prompt reaches 100% arithmetic execution on Qwen3-0.6B and 93.8% on Qwen3-1.7B. Capability changes the error type: 0.6B falsely blocks two controls, while 1.7B has no false block and one arithmetic miss. A clean 4B smoke supports feasibility but not scaling. Paper 1 therefore establishes a plausible few-shot semantic execution interface and a protocol-only adapter result across three small-model families. Adapter+minimal reaches perfect execution and observed control safety in all three; it reaches perfect answers in Qwen and SmolLM and 15/16 in Gemma, where residual ICL restores 16/16. Base+minimal never opens a valid arithmetic block in any family. Context enables cold-start behavior but costs more tokens and can produce false interventions; after adaptation it is redundant or harmful except for the measured Gemma residual. The normalized runtime is training-free but remains limited by model expression exposure and result integration. Gemma and SmolLM oracle runs further show that exact reinjection alone does not guarantee result use. The resulting architecture is hybrid: context for rapid development and measured residuals, weights for a stable semantic contract, and the runtime for typed parsing, bounds, exact computation, and enforced use. Powered confirmatory data and an authorized Llama run remain necessary before stronger placement or family claims. On public GSM8K, however, this narrow adapter and the training-free automatic interface do not transfer: high activation coexists with 6.5% or lower operation recall and at most 3.3% accuracy, while the bounded-calculator oracle reaches 75.8%. Where interface knowledge should live is therefore conditional: syntax can live in context or weights and exactness in the runtime, but natural-language decomposition is not solved by any of the three in this experiment. The ASL bootstrap turns that negative result into a testable next stage without claiming success: public records and scopes are mined, a compact extensible surface parses to typed state, and all 150 Q1 semantic programs execute exactly. A frozen grouped pilot now tests whether context, weights, or controlled augmentation produce a measurable compilation signal. Context alone teaches the surface, 50 genuinely different origi-

nals improve semantic compilation, and three residual examples complement the 100-row adapter. Controlled perturbations mainly improve invariance, not untouched semantic generalization. The result supports a hybrid placement—context for local specialization, weights for a learned ASL contract, and the runtime for lowering, types, and deterministic execution—but its untouched 25-program test and non-manual Q1 labels deliberately bound the conclusion. The relation-diversified 500-original adapter improves to 11/25, but clause-local executed-state compilation regresses to 7/25 and oracle state reaches only 6/25. Full-question local compilation recovers 11/25 but does not exceed whole-program synthesis. Valid deltas remain semantically weak, particularly on multi-clause GSM8K. Fine-grained replay localizes the strict whole-program deficit to attribute grounding (.144 F1), dependency preservation (.113 F1), relation direction (12/30), and RETURN grounding (7/25), despite .695 operator F1; 6/20 state-inequivalent programs still obtain the correct answer. The next gate is therefore matched adapter-capacity and placement diagnosis rather than recovery from poisoned runtime state.

A Deterministic Semantic Diagnostic Taxonomy

The replay analyzer emits multi-label records; categories are not mutually exclusive. Path decomposition treats the first path component as the entity, non-qualifier semantic tokens as attributes, and separately records temporal, cardinality, status, and grouping qualifiers. CCIR assignments provide target, ordered sources, root and nested operators, constants, directed edges, and the RETURN expression. This supports separate tests for participants, direction, noncommutative argument order, literal magnitude, inter- versus intra-object relations, direct source-fact attachment, premature literal collapse, and transitive shortcuts. Aggregation and temporal supports are reported only when the reference contains the corresponding deterministic cues.

Standalone alignment is one-to-one and requires compatible entities, lexical semantic overlap, and consistent graph roles; ambiguity or an unresolved local ontology enters the judge queue. Stateful replay instead requires exact reuse of already established paths and records accidental renames, duplicate slots, and unresolved references. The optional semantic judge is not run in the reported results. Coreference target accuracy requires mention annotations that the frozen corpus does not contain, and unit compatibility requires a dimensional type system; both are marked not measurable rather than inferred from generic parse or type success. Regression fixtures cover wrong entities and attributes, qualifier-only mismatches, allowed standalone versus forbidden stateful renames, relation and argument reversal, wrong constants, literal collapse, transitive shortcuts, each/total confusion, temporal direction, RETURN grounding, and multi-label failures without remote calls.

References

[1] T. Schick, J. Dwivedi-Yu, R. Dessi, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom. Toolformer: Language models can teach themselves to use tools. *NeurIPS*, 2023. https://arxiv.org/abs/2302.04761.

[2] L. Gao, A. Madaan, S. Zhou, U. Alon, P. Liu, Y. Yang, J. Callan, and G. Neubig. PAL: Program-aided language models. *ICML*, 2023. https://arxiv.org/abs/2211.10435.

[3] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. *ICLR*, 2023. https://arxiv.org/abs/2210.03629.

[4] E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen. LoRA: Low-rank adaptation of large language models. *ICLR*, 2022. https://arxiv.org/abs/2106.09685.

[5] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. *ICLR*, 2024. https://arxiv.org/abs/2310.11511.

[6] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv:2110.14168*, 2021. https://arxiv.org/abs/2110.14168.